CS 147: Lab 04
Due Date: October 20, 2026

September 3, 2026

This homework assignment requires writing assembly tests for your project. We will be releasing these tests to help everyone debug their processors, so I strongly recommend you add comments to make it easy for your fellow classmates to understand what your tests do. Here are some links to important documents/webpages you will need to properly complete this assignment:

1. Info about how to write assembly code and also about how to use the assembler can be found in the Using the assembler page.
2. Details about what each instruction means is available in the ISA specification.
3. The WISC-SJ26 Simulator-Debugger page has information on the simulator for the WISC-SJ26 ISA.
4. Other useful documents on how to write tests:
   (a) Other example test programs to understand syntax etc.
   (b) Test Programs FAQ

The work flow we will follow is:

1. Write test in WISC-SJ26 assembly language.
2. Assemble using:

```
./run assemble assignments/hw04/<subproblem>/<test_name>.asm
   assignments/hw04/<subproblem>/<test_name> > assignments/hw04/<
   subproblem>/<test_name>_all.img
```

3. Simulate the test in the simulator and make sure your test is doing what you thought it was doing. Use the simulator via:

```
./run simulate assignments/hw04/<subproblem>/<test_name>_all.img
```

4. Run the integrated testbench checks for both subproblems:

```
./run test hw4 hw4_1
./run test hw4 hw4_2
```

5. Write up your explanation of each test.

Here is a short demo:

```
prompt% ./run assemble assignments/hw04/hw4_1/add_0.asm assignments/
   hw04/hw4_1/add_0 > assignments/hw04/hw4_1/add_0_all.img
Created the following files
add_0_0.img add_0_1.img add_0_2.img add_0_3.img
add_0_all.img add_0.lst

prompt% ./run simulate assignments/hw04/hw4_1/add_0_all.img

WISCalculator v1.0
Author Derek Hower (drh5@cs.wisc.edu)
Type "help" for more information
Loading program...
Executing...
lbi r1, -1
INUM: 0 PC: 0x0000 REG: 1 VALUE: 0xffff
lbi r2, -1
INUM: 1 PC: 0x0002 REG: 2 VALUE: 0xffff
add r3, r1, r2
INUM: 2 PC: 0x0004 REG: 3 VALUE: 0xfffe
halt
program halted
INUM: 3 PC: 0x0006
Program Finished

prompt%
```

The simulator will print a trace of each instruction along with the state of the relevant registers. You should examine these to make sure that your test is indeed doing what is expected.

Suggestions:

- Identify corner cases. Think about possible bugs in the hardware with pipelining.
- Ideally tests should be short and target specific cases, NOT all cases at once.
- Write comments in your assembly code explain that what the test is doing (comments use "//" for our assembler)
- Make sure that all of your assembly files will assemble. You won't get credit for malformed assembly.

- The goal of this problem is to make sure you understand the ISA and develop targeted tests for the (pipelined) hardware. Understanding the ISA is required (and extremely helpful!) before building (pipelined) hardware for it!

Problem 1 (20 points)

1. (10 points) Generate your assigned Problem 1 target opcodes:

   ```
   ./run generate hw4
   ```

   This creates:

   ```
   assignments/hw04/hw4_1/targets.txt
   ```

   Target letters map to opcodes as follows:

Letter	Instruction
A	ROLI
B	SLLI
C	RORI
D	SRLI
E	ST
F	LD
G	STU
H	BTR
I	ROL
J	SRL
K	ROR
L	SLL
M	SEQ
N	SLT
O	SLE
Q	SCO
R	SIIC, RTI
S	SLBI
T	JR
U	JAL
V	JALR
W	XOR, XORI
X	ANDN, ANDNI

Write three unique tests (`p1-1.asm`, `p1-2.asm`, `p1-3.asm`) using the assigned targets in order. In addition to using each assigned instruction, your tests should exercise different parts of the pipeline (for example, forwarding paths, stalls, or corner cases such as sign extension).

It is fine if your tests contain other instructions (e.g., LBI to initialize registers, HALT to stop the program, and NOPs to test various pipeline stalls, forwarding cases, or branch cases), but each of your tests must include its assigned target instruction.

2. (10 points) Write an explanation of your programs including where and why forwarding takes place. Write your answer in `hw4_1/p1_answers.txt` under:

   ```
   HW4_1_2 Answer:
   // Your answer here
   ```

   (Explain all three programs in the same answer block.)

Problem 2 (20 points)

1. (10 points) Write two unique assembly programs to demonstrate why branch prediction is necessary and useful. Write your code in `p2-1.asm` and `p2-2.asm`, respectively.

2. (5 points) Write an explanation of your programs and how branch prediction helps in `hw4_2/p2_answers.txt` under:

   ```
   HW4_2_2 Answer:
   // Your answer here
   ```

3. (5 points) Will branch prediction always take only 1 cycle in your 5-stage processor? Write your answer in `hw4_2/p2_answers.txt` under:

   ```
   HW4_2_3 Answer:
   // Your answer here
   ```

Submission Instructions

To build a submission for grading, run:

- `./run build_submission <assignment>`
- This will automatically run `./run test <assignment>`, then run the generated submission through the integrated autograder.
- A submission ZIP is created in `generated_turnins/<assignment>/`.

- If you run `build_submission` multiple times for the same assignment, the generated zip files will be prepended with a number. The most recent submission should be the item with the highest number.
- **IMPORTANT:** Running `./run build_submission <assignment>` **DOES NOT** actively submit your assignment, it only generates the file you will submit. This file must be manually uploaded to Gradescope.

If an assembly file is missing or does not compile, you will automatically get zero points for that part.

Note: no schematics are needed for this assignment.